

Investigación: Temas de Desarrollo Web

Errores HTTP · Swagger en Express · Principios SOLID · Patrones
Arquitectónicos

Universidad del Rosario — Abril 2026

Este documento es una investigación de nivel introductorio sobre cuatro temas que aparecen constantemente en el desarrollo web moderno. La idea no es cubrir todo a fondo sino entender los conceptos básicos con ejemplos concretos y lenguaje directo. Los temas son: los códigos de estado HTTP y qué significan, cómo documentar una API en Express usando Swagger, los cinco principios SOLID de diseño orientado a objetos, y los principales patrones arquitectónicos que se usan hoy en día.

1. Códigos de Estado HTTP

Cuando el navegador (o cualquier cliente) le hace una petición a un servidor web, el servidor siempre responde con un número de tres dígitos llamado código de estado HTTP. Ese número no es decoración: le dice al cliente exactamente qué pasó con su petición. Entender estos códigos es fundamental para depurar errores, diseñar APIs y entender cómo se comunican los sistemas en internet.

Los códigos se agrupan en cinco familias según el primer dígito. Los códigos 1xx son informativos y significan que el servidor recibió la petición y sigue procesando. Los 2xx indican éxito: la petición se completó correctamente. Los 3xx son redirecciones, lo que quiere decir que el recurso se movió a otro lado y el cliente debe ir a buscarlo allá. Los 4xx son errores del cliente, es decir, el problema lo causó quien hizo la petición. Y los 5xx son errores del servidor, donde el problema está del lado del backend.

Códigos de éxito (2xx)

El **200 OK** es el más común de todos. Significa que la petición salió perfecta y el servidor está devolviendo el recurso solicitado. Cuando una API devuelve datos correctamente, casi siempre viene acompañada de un 200. El **201 Created** se usa cuando se creó un recurso nuevo exitosamente, por ejemplo al registrar un usuario nuevo con un POST. El **204 No Content** indica que todo salió bien pero el servidor no tiene nada que devolver, lo cual es típico en operaciones DELETE exitosas donde ya no hay recurso que mostrar.

Códigos de redirección (3xx)

El **301 Moved Permanently** indica que el recurso cambió de dirección de forma permanente. Los navegadores y motores de búsqueda lo interpretan como una redirección definitiva y actualizan sus registros. Es muy usado en migraciones de sitios web y en SEO. El **302 Found** es una redirección temporal: el recurso está en otro lugar por ahora, pero puede volver. El **304 Not Modified** es especial: le dice al cliente que el recurso no cambió desde la última vez que lo pidió, así que puede usar la versión en caché que ya tiene guardada, lo cual ahorra ancho de banda.

Errores del cliente (4xx)

El **400 Bad Request** significa que la petición está mal armada. Puede ser un JSON malformado, campos obligatorios que faltan o parámetros con un formato incorrecto. El error es del cliente. El **401 Unauthorized** indica que el cliente no está autenticado:

necesita iniciar sesión o enviar un token válido para poder continuar. El **403 Forbidden** es diferente al 401: acá el cliente sí está autenticado, pero no tiene permiso para hacer lo que está intentando. La diferencia es sutil pero importante: 401 es 'no sé quién sos', 403 es 'sé quién sos pero no podés entrar'.

El **404 Not Found** es probablemente el más famoso de todos. Significa que el recurso no existe en esa URL. Puede ser porque se escribió mal la dirección, porque el recurso fue eliminado o porque nunca existió. El **405 Method Not Allowed** aparece cuando se usa el método HTTP incorrecto, por ejemplo intentar un DELETE en una ruta que solo acepta GET. El **409 Conflict** surge cuando hay un conflicto con el estado actual del recurso, como intentar crear un usuario con un email que ya existe en la base de datos. El **422 Unprocessable Entity** es similar al 400 pero más específico: el servidor entendió la petición pero no puede procesarla porque los datos no pasan la validación. El **429 Too Many Requests** indica que el cliente hizo demasiadas peticiones en poco tiempo y el servidor activó el rate limiting.

Errores del servidor (5xx)

El **500 Internal Server Error** es el error genérico del servidor. Significa que algo falló adentro del backend, generalmente una excepción no manejada en el código. Cuando aparece en producción es señal de que algo está muy mal y hay que revisar los logs. El **502 Bad Gateway** ocurre cuando un servidor intermediario (como un proxy o un balanceador de carga) recibe una respuesta inválida del servidor al que le estaba preguntando. Es común en arquitecturas con múltiples servicios. El **503 Service Unavailable** significa que el servidor no puede atender peticiones en este momento, ya sea porque está sobrecargado o en mantenimiento.

2. Documentación con Swagger en Express

Swagger, hoy estandarizado bajo el nombre OpenAPI, es una especificación y conjunto de herramientas que permite documentar APIs de forma estructurada y visual. En lugar de escribir un documento Word explicando los endpoints, Swagger genera una interfaz web interactiva donde cualquier persona puede ver qué rutas existen, qué parámetros reciben, qué devuelven y hasta probarlas en vivo desde el navegador. Es un estándar ampliamente adoptado en la industria.

La principal ventaja de integrar Swagger en un proyecto Express es que la documentación vive junto al código fuente. Si un endpoint cambia, el desarrollador actualiza el comentario encima de la ruta y la documentación se actualiza automáticamente. Esto elimina el problema clásico de tener una documentación desactualizada que ya no refleja cómo funciona realmente la API. Además, facilita la comunicación entre equipos: el equipo de frontend puede consultar la documentación sin tener que preguntarle al backend qué esperar de cada llamada.

Instalación y configuración básica

Para usar Swagger en un proyecto Express se necesitan dos paquetes: **swagger-jsdoc**, que se encarga de leer los comentarios del código y convertirlos en una especificación OpenAPI, y **swagger-ui-express**, que sirve la interfaz visual en una ruta de la aplicación. Se instalan con el siguiente comando:

```
npm install swagger-jsdoc swagger-ui-express
```

Una vez instalados, se crea un archivo de configuración (o se agrega directamente en `app.js`) donde se define la información básica de la API y se le indica a `swagger-jsdoc` en qué archivos debe buscar los comentarios de documentación:

```
const swaggerJsdoc = require('swagger-jsdoc'); const swaggerUi =
require('swagger-ui-express'); const options = { definition: { openapi:
'3.0.0', info: { title: 'Mi API', version: '1.0.0', description:
'Documentacion de mi API', }, servers: [{ url: 'http://localhost:3000' }], },
apis: ['./routes/*.js'], }; const specs = swaggerJsdoc(options);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(specs));
```

Con esa configuración, al correr el servidor y entrar a la ruta `/api-docs` desde el navegador, ya aparece la interfaz visual de Swagger con toda la documentación generada automáticamente.

Documentar un endpoint

La documentación de cada endpoint se escribe en un comentario especial justo encima de la ruta, usando la sintaxis de JSDoc con la etiqueta @swagger. Dentro de ese comentario se escribe YAML con la descripción del endpoint, sus parámetros y las posibles respuestas:

```
/** * @swagger * /usuarios: * get: * summary: Obtiene todos los usuarios * tags: [Usuarios] * responses: * 200: * description: Lista de usuarios obtenida correctamente * 500: * description: Error interno del servidor */  
router.get('/usuarios', async (req, res) => { const usuarios = await  
  Usuario.findAll(); res.status(200).json(usuarios); });
```

Para endpoints que reciben parámetros en la URL se usa la sección parameters con in: path. Para endpoints que reciben un cuerpo JSON se usa requestBody con el schema del objeto esperado. Esto permite que Swagger muestre exactamente qué estructura debe tener cada petición y qué puede devolver el servidor en cada escenario posible, incluyendo los códigos de error.

3. Principios SOLID

SOLID es un acrónimo formado por las iniciales de cinco principios de diseño orientado a objetos propuestos por Robert C. Martin, conocido en la comunidad como Uncle Bob. Estos principios no son reglas absolutas ni un checklist que hay que seguir en todos los proyectos, sino guías que ayudan a escribir código más fácil de mantener, de extender y de entender con el tiempo. Aplicarlos bien hace que agregar funcionalidad nueva no implique romper lo que ya funcionaba.

S — Single Responsibility Principle

El Principio de Responsabilidad Única dice que una clase o función debe tener una sola razón para cambiar. Dicho de otra forma, cada módulo del código debe encargarse de una sola cosa. Si una clase hace demasiadas cosas a la vez, cualquier cambio en una de ellas puede romper las otras por accidente.

Un ejemplo clásico de violación es una clase Usuario que al mismo tiempo guarda datos en la base de datos, envía correos de bienvenida y genera reportes en PDF. Si cambia el proveedor de email, hay que tocar la clase Usuario aunque eso no tenga nada que ver con los usuarios en sí. La solución es separar: UsuarioRepository para la base de datos, EmailService para los correos y ReporteService para los reportes.

O — Open/Closed Principle

El Principio Abierto/Cerrado establece que el código debe estar abierto para extenderse pero cerrado para modificarse. Esto significa que se puede agregar nueva funcionalidad sin tener que cambiar el código que ya existe y que ya funciona. La forma típica de lograrlo es usando abstracciones como interfaces o clases base.

Si se tiene un sistema de pagos que soporta tarjeta de crédito y se quiere agregar soporte para PayPal, la solución correcta no es entrar a la clase de pagos existente y agregarle un if para PayPal. La solución es crear una nueva clase PayPalPago que implementa la misma interfaz que TarjetaPago, sin tocar nada del código anterior.

L — Liskov Substitution Principle

El Principio de Sustitución de Liskov dice que si una clase B hereda de una clase A, entonces en cualquier lugar donde se usa A debería poder usarse B sin que el programa falle ni se comporte de manera inesperada. Básicamente, las subclases deben respetar el

contrato que establece la clase padre.

El ejemplo más citado es el del Pato de Plástico. Si Ave tiene un método volar() y Pato hereda de Ave, todo bien. Pero si se crea PatoDePlastico que también hereda de Ave y ese método lanza una excepción porque los patos de plástico no vuelan, se está rompiendo este principio. La jerarquía de herencia está mal diseñada desde el principio.

I — Interface Segregation Principle

El Principio de Segregación de Interfaces indica que es mejor tener varias interfaces pequeñas y específicas que una sola interfaz grande y general. Si una clase se ve obligada a implementar métodos que no necesita solo porque la interfaz los incluye, algo está mal en el diseño.

Por ejemplo, si se tiene una interfaz Trabajador con métodos como trabajar(), comer() y dormir(), un robot que implementa esa interfaz se ve forzado a implementar comer() y dormir() aunque no tenga sentido para él. La solución es separar en interfaces más pequeñas: Trabajador con trabajar(), y SerVivo con comer() y dormir(), para que cada clase implemente solo lo que le corresponde.

D — Dependency Inversion Principle

El Principio de Inversión de Dependencias dice que los módulos de alto nivel no deben depender directamente de los módulos de bajo nivel. Ambos deben depender de abstracciones. Esto evita el acoplamiento fuerte entre capas y hace que el sistema sea más fácil de cambiar.

Si un servicio de pedidos importa directamente MySQLDatabase, está acoplado a esa tecnología específica. Si el día de mañana se quiere cambiar a MongoDB, hay que reescribir el servicio. La solución es que el servicio dependa de una interfaz DatabaseRepository, y que MySQLDatabase o MongoRepository la implementen. Así el servicio no sabe ni le importa qué base de datos se está usando debajo.

4. Patrones Arquitectónicos

Un patrón arquitectónico es una solución probada y documentada para organizar el código de una aplicación a gran escala. No se trata de fragmentos de código reutilizables sino de decisiones estructurales sobre cómo se dividen las responsabilidades del sistema y cómo se comunican sus partes. Elegir bien el patrón desde el principio puede marcar la diferencia entre un sistema que escala bien y uno que se vuelve imposible de mantener.

MVC — Model, View, Controller

MVC es el patrón más clásico y el primero que se aprende en la mayoría de los cursos de desarrollo web. Divide la aplicación en tres capas con responsabilidades claras. El Model maneja los datos y la lógica de negocio, es quien habla con la base de datos. La View es lo que ve el usuario: el HTML, los templates o los componentes de interfaz. El Controller recibe las peticiones del usuario, le pide al Model los datos que necesita y se los manda a la View para que los muestre.

La gran ventaja de MVC es que separa claramente las tres responsabilidades, lo que facilita el trabajo en equipo: un desarrollador puede trabajar en la vista sin tocar la lógica de negocio y viceversa. Frameworks como Express con EJS, Laravel, Ruby on Rails y Spring MVC están basados en este patrón.

MVP — Model, View, Presenter

MVP es una variante de MVC que se popularizó especialmente en el desarrollo de aplicaciones móviles Android. La diferencia principal está en el rol de la View: en MVP la Vista es completamente pasiva, solo muestra lo que le mandan y no toma ninguna decisión. Toda la lógica de presentación se mueve al Presenter, que actúa de puente entre el Model y la View. El Presenter obtiene los datos del Model, los procesa y le dice a la View exactamente qué mostrar. Esto hace que la lógica sea mucho más fácil de probar con unit tests porque no depende de la interfaz gráfica.

MVVM — Model, View, ViewModel

MVVM es el patrón favorito del desarrollo frontend moderno y de las aplicaciones móviles actuales. Su característica principal es el data binding: la View se suscribe al ViewModel y se actualiza automáticamente cuando los datos cambian, sin que nadie tenga que llamar explícitamente a ningún método. El ViewModel expone los datos en un formato que la View puede consumir directamente y también recibe las acciones del usuario para

procesarlas.

Frameworks como Vue.js, Angular y herramientas como Jetpack Compose en Android o SwiftUI en iOS están contruidos sobre este patrón. La reactividad que ofrecen estos frameworks es justamente la implementación del data binding que define MVVM.

Arquitectura de Microservicios

La arquitectura de microservicios es la alternativa moderna al monolito tradicional. En lugar de tener una sola aplicación grande que hace todo, el sistema se divide en servicios pequeños e independientes, donde cada uno tiene su propia base de datos, puede estar escrito en un lenguaje diferente y se puede desplegar de forma independiente. Los servicios se comunican entre sí a través de APIs REST o mensajes asíncronos.

La principal ventaja es la escalabilidad: si el servicio de pagos está sobrecargado, se pueden levantar más instancias solo de ese servicio sin tocar el resto. La desventaja es la complejidad operacional: hay que manejar la comunicación entre servicios, el monitoreo distribuido, la consistencia de datos entre bases de datos diferentes y herramientas de orquestación como Kubernetes. Empresas como Netflix, Amazon y Uber usan esta arquitectura a gran escala.

Arquitectura Hexagonal (Ports and Adapters)

La arquitectura hexagonal, también conocida como Clean Architecture o Ports and Adapters, fue propuesta por Alistair Cockburn. La idea central es que el núcleo de la aplicación, es decir la lógica de negocio pura, no debe depender de ningún detalle externo como la base de datos, el framework web o la interfaz de usuario. Esos son detalles intercambiables que rodean al núcleo.

El Dominio contiene las entidades y reglas de negocio sin ningún import de tecnología específica. Los Puertos son interfaces que definen cómo el mundo exterior puede interactuar con el dominio. Los Adaptadores son las implementaciones concretas de esas interfaces: el controlador REST, el repositorio de MySQL, el cliente de Stripe para pagos. Si mañana se quiere cambiar MySQL por MongoDB, solo se escribe un nuevo adaptador sin tocar nada del dominio. Esta arquitectura es muy usada en proyectos empresariales grandes que siguen Domain-Driven Design (DDD).

En resumen, los patrones arquitectónicos no son dogmas que hay que seguir al pie de la letra. Son herramientas con contextos donde funcionan mejor. Para empezar, MVC ya da una base sólida y suficiente. A medida que los proyectos crezcan en complejidad, tamaño de equipo o requerimientos de escalabilidad, los patrones más avanzados se vuelven no solo útiles sino necesarios.